

Port OpenWrt su Nubia RedMagic 7 (NX679J)

Rapporto di analisi del repository *openwrt-nx679j*: igiene, riproducibilità, fasi, agenti e modelli AI, attribuzione umano/AI e stima dei costi

Repository analizzati: github.com/CAPP-TESTS/openwrt-nx679j e github.com/Saddytech/openwrt-nx679j (commit f9a92fc)

Data del rapporto: 4 ottobre 2026

Metodo: clonazione e analisi locale dell'intero albero (2.520 file), più verifica dei prezzi pubblici dei modelli

Sintesi

Il repository documenta un port di OpenWrt 25.12.5 sul Nubia RedMagic 7 (NX679J, Snapdragon 8 Gen 1 / SM8450). Il port non è un OpenWrt “nativo” nel senso stretto: è un userland OpenWrt eseguito in chroot sullo slot B, sopra il kernel vendor Android 5.10.66 lasciato intatto. Il modem X65 è pilotato direttamente via QMI/QRTR, senza ModemManager, e l'interfaccia è una UI scritta in C su DRM/KMS.

Il lavoro è stato svolto quasi interamente da agenti AI, con un ruolo umano di direzione, supervisione e intervento fisico sul dispositivo. I risultati principali dell'analisi sono i seguenti.

- **Igiene buona, riproducibilità debole.** Nessun dato personale sensibile esposto e segreti correttamente redatti. Mancano però gli script delle build v91-v180, tutti gli script presenti usano percorsi assoluti della macchina dell'autore, e non ci sono procedure per il rootfs OpenWrt né per il flash.
- **Quattro fasi.** Una fase pre-Hermes (2 luglio – 7 agosto) di provenienza ignota, una fase di test dei modelli (4-15 settembre), la fase Hermes vera e propria (16-29 settembre, circa 180 iterazioni di build) e la pubblicazione (30 settembre).
- **Non solo Hermes + DeepSeek.** DeepSeek non compare in nessun file. Sono documentati GPT-6 Astra (consulente di design e di diagnosi), gpt-5.6-luna (test), claude-haiku-4-5 (tentativo fallito) e OpenRouter (fallito). L'analisi visiva delle immagini richiede un modello con input visivo, diverso da DeepSeek V4 Pro. La fase P0 porta l'impronta stilistica di un agente diverso, non identificabile.
- **Attribuzione.** Agentico il 97-99,5% del volume prodotto e il 93-98% dell'esecuzione. Sulla direzione delle svolte decisive, invece, la quota umana stimata è del 25-40%.
- **Costi stimati.** Circa 67 dollari nello scenario minimo, 176 nel medio e 523 nel massimo. La stima più realistica è di circa 190-200 dollari, con una forchetta difendibile da circa 35 a circa 800 dollari. Se il modello principale fosse GPT-6 Astra invece di DeepSeek, il solo loop principale costerebbe circa 2.500 dollari.

Nota di metodo. La storia git è stata appiattita in un unico commit e i log di sessione di Hermes (state.db, request_dump_*.json) non sono pubblicati. Tutte le attribuzioni e i volumi sono quindi ricostruiti dal contenuto dei file. I numeri vanno letti come stime motivate, non come misure: ogni stima è accompagnata dalle ipotesi da cui dipende.

1. Il progetto

Obiettivo dichiarato. Avviare OpenWrt su uno smartphone Android di fascia alta, ottenere un canale di controllo persistente sull'unica porta USB-C, usare il modem cellulare come uplink e mantenere sempre una via di recupero: lo slot A con Android stock e Magisk, oppure la modalità EDL di Qualcomm. Il README lo presenta come un “port completo” di OpenWrt 25.12.5.

Architettura effettiva. Nei documenti la parola “nativo” significa che OpenWrt gira direttamente sull'hardware senza Android in esecuzione, ma il kernel resta quello del produttore. Il progetto non porta un kernel proprio o mainline. Il tentativo con un kernel upstream 6.6.y della fase P0 è fallito: si bloccava dopo 15 secondi per la mancanza del supporto all'hypervisor Gunyah. Il risultato finale è quindi un userland OpenWrt innestato sulla piattaforma Android esistente.

Componente	Soluzione adottata	Note
Kernel	Vendor 5.10.66 (android12), intatto	Moduli vendor caricati dal ramdisk, nessuna patch al kernel
Avvio	Ramdisk custom nella boot image dello slot B (header v4)	PID 1 e init propri (init v8/v9), gadget USB NCM su 10.0.0.1
Userland	OpenWrt 25.12.5 (r33051-f5dae5ece4), armsr/armv8, musl	Eseguito in chroot, con LuCI, SSH e persistenza dentro il ramdisk
Modem	Qualcomm X65 via QMI su QRTR (AF_QIPCRTR)	ModemManager dismesso dalla v130: catena propria, plugin LuCI e supervisore del collegamento
Wi-Fi	QCA6490 / WCN6855 (cnss2)	Access point NX679J-TEST a 5 GHz, NAT verso la WAN cellulare
Display e input	UI nativa in C su DRM/KMS, commit atomici continui	Il pannello collassa senza commit per più di ~58 ms; navigazione con i tasti laterali
Recupero	Slot A Android + Magisk, fastboot, EDL	Il cambio di slot è autonomo tramite un binario di reboot e fastboot

Identità dei repository. CAPP-TESTS/openwrt-nx679j e Saddytech/openwrt-nx679j contengono lo stesso identico commit (f9a92fc, 30 settembre 2026 15:46 +0200, messaggio “first commit”, autore Saddytech). La release v1.0 è stata pubblicata il 1° ottobre. Il repository di CAPP-TESTS va quindi considerato una copia, o un fork senza modifiche, di quello di Saddytech.

Stato dichiarato nel tempo. Il report tecnico del 20 settembre dichiarava esplicitamente “Internet NON ancora raggiunto”: il primo ping cellulare provocava un reboot. I documenti successivi registrano la connessione dati con il proto standard (22 settembre), l'eliminazione del limite orario del bearer dopo 72 minuti di verifica (24 settembre), il modem senza ModemManager (v130) e lo standby definitivo del display (v180, 25 settembre). Queste affermazioni sono documentate ma non verificabili indipendentemente dal repository, perché mancano le immagini e le build finali (vedi sezione 2).

Struttura del repository.

Cartella	Contenuto
01-documenti	Handoff, prompt di sessione, report, istantanee di STATO-ATTUALE e DOCUMENTAZIONE
02-sorgenti	Init v8/v9, relay/worker, build-v54...v90.py, UI in C, patch, configurazioni live
04-persistenza	etc.tar, luci.tar, tools.tar sanificati, come iniettati nel ramdisk
05-bootchain-re	Reverse engineering della catena di boot, baseline nativa, bootloader-re (fase P0), ricerca su una CVE
06-ricerche	port-work (fase P0), ricerche su DSI/display, idlepc

Cartella	Contenuto
07-evidenze-runtime	Proprietà, dmesg, GPIO e regolatori raccolti da Android l'11 luglio
08-skill	18 skill Hermes (metodo, diagnosi, regole dell'utente, fan-out di ricerca)
09-albero-originale	Resto dell'albero di lavoro: EDL, build headless, hypervisor-bypass, log dei subagenti

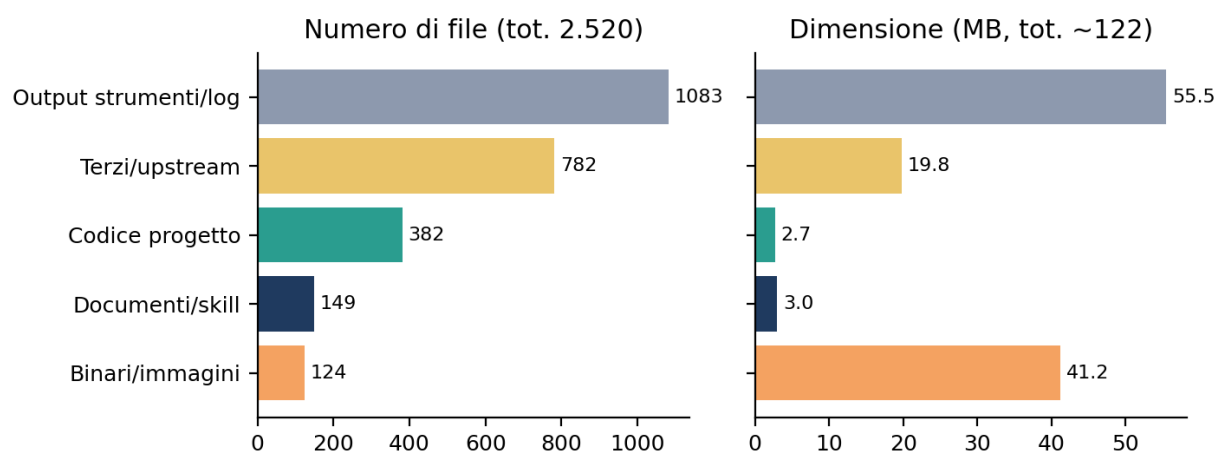


Figura 1 — Composizione del repository per tipo di file. Solo il 21% dei file è materiale scritto dal progetto (codice e prosa).

2. Igiene del repository e riproducibilità

2.1 Igiene

Su questo fronte il repository è curato. Il verificatore automatico dell'autore dichiara 23 controlli su 23 superati: segreti cercati anche dentro binari e archivi tar, dati personali, artefatti vendor riconosciuti per estensione e per magic byte, coerenza dei numeri dichiarati. Gitleaks segnala 2 risultati, documentati come falsi positivi. Ho verificato in modo indipendente i punti più critici.

Controllo	Esito	Dettaglio
Identificativi radio (IMEI, IMSI 22288..., ICCID 89..., MSISDN +39)	Nessuna occorrenza	Ricerca per pattern sull'intero albero
Redazioni	≈ 9.000 marcature	Numero di serie mascherato (3dbd****), chiave Wi-Fi [REDACTED]
Credenziali OpenWrt (etc.tar)	Pulite	Password di root bloccata (!), authorized_keys sanificato
Indirizzi IP	Solo privati	10.0.0.1 (gadget USB), 10.0.2.x (QEMU), un indirizzo CGNAT dell'operatore
Licenze	Presenti	MIT per il codice, CC BY 4.0 per i documenti, LICENSES/ e THIRD_PARTY_LICENSES.md
Artefatti proprietari	Esclusi per scelta	Firmware, moduli .ko, boot image e loader Firehose non sono inclusi

Ci sono però anche dei limiti che riguardano la trasparenza più che la sicurezza.

- **Storia appiattita.** Un solo commit con 2.520 file: è impossibile ricostruire chi ha cambiato cosa e quando.
- **Deduplicazione invasiva.** DEDUPLICA.md dichiara la rimozione di 1.450 file (circa 59 MB) da 239 cartelle. Per alcuni file indica come copia canonica un "archivio di revisione" che non è pubblicato, per esempio diverse versioni di STATO-ATTUALE.md, i file ESITO e quelli DECISIONE.
- **La verifica riguarda l'igiene, non il funzionamento.** Nessuno dei 23 controlli dimostra che il port si possa ricostruire o avviare.

2.2 Riproducibilità

Il README afferma che tutti gli artefatti esclusi "si ricostruiscono" e che ESTRAZIONE-BLOB.md contiene procedure "complete e verificate". La verifica dà un risultato più sfumato: le procedure per estrarre i blob vendor ci sono, ma il resto della catena è incompleto.

Requisito per ricostruire	Stato nel repo	Valutazione
Firmware stock, partizioni, moduli, touch, DTB/DTBO	Procedure presenti (sezioni 1-6 di ESTRAZIONE-BLOB.md, 234 righe)	Sufficiente per un esperto
Loader Firehose, GPT, EFS, sorgenti GPL	Procedure presenti (sezioni 7-10)	Sufficiente; l'EDL però "funzionava" solo in passato
Rootfs OpenWrt 25.12.5	Nessuna procedura in ESTRAZIONE-BLOB (0 occorrenze); solo l'URL di origine in DOCUMENTAZIONE	Lacuna
Script di build delle immagini	38 script: v10 e v54-v90. Mancano v91-v180, e la versione finale documentata è v180	Lacuna grave
Portabilità degli script	Tutti i 38 contengono percorsi assoluti /home/user/nx679j-stock/...	Da adattare a mano
Procedura di flash	0 occorrenze di comandi di flash in README ed ESTRAZIONE-BLOB	Lacuna

Requisito per ricostruire	Stato nel repo	Valutazione
Persistenza (etc/luci/tools)	Presente, in forma sanificata	Utilizzabile
Sorgenti della UI, init, relay, catena QMI	Presenti (nx679j-ui.c, init v9, qmi-qtr.c, ...)	Utilizzabili
Immagini pronte o rootfs	Assenti per scelta	Coerente con la licenza, ma non avviabile così com'è

Giudizio. Il port **non è riproducibile “chiavi in mano”**. Un reverse engineer esperto, in possesso del dispositivo, potrebbe probabilmente ricostruire una versione intorno alla v90 partendo da build-v90.py: dovrebbe procurarsi i blob con ESTRAZIONE-BLOB, scaricare il rootfs OpenWrt, correggere i percorsi e ricavare la procedura di flash dai log. Lo stato finale descritto nella documentazione (v180, con UI completa e modem senza ModemManager) non è invece ricostruibile dal solo repository, perché mancano i builder delle ultime 90 iterazioni.

3. Fasi delle attività

La datazione non può basarsi sui metadati: git contiene un solo commit e tutti i file hanno la stessa data. Per circa metà dei file ho ricavato la data dal percorso o dal contenuto (pattern come 20260917 o 2026-07-10). Gli altri li ho assegnati in base alla cartella, seguendo la cronologia che DOCUMENTAZIONE.md stessa ricostruisce. La precisione è quindi buona a livello di fase, non di singolo giorno.

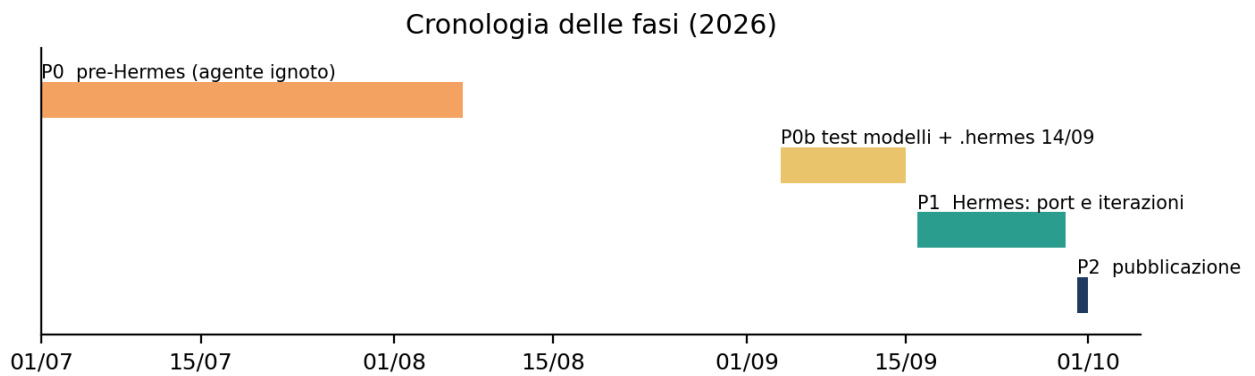


Figura 2 — Cronologia delle fasi.

Fase	Periodo	Attività principali	Agente
P0	2 luglio - 7 agosto (più l'handoff del 16/09 alle 10:30)	Download dell'OTA V311, patch Magisk, backup dello slot B, prototipi OpenWrt fasi 1-4, moduli stock, readback EDL del 17/07, RE del bootloader e di Gunyah, tentativi con kernel upstream 6.6.y falliti	Ignoto; non è Hermes
P0b	4-15 settembre	04/09: 7 sessioni di test di modelli e provider (gpt-5.6-luna, claude-haiku-4-5 fallito). 14/09: cartella .hermes in hypervisor-bypass, senza sessione registrata	Hermes (configurazione)
P1	16-29 settembre	Catena di boot, init/PID 1, gadget USB, chroot OpenWrt, QMI/QRTR, Wi-Fi, LuCI, UI su DRM/KMS, standby, ottimizzazione dei tempi di boot: v10 → v180	Hermes + subagenti + GPT-6 Astra + modello di visione
P2	30 settembre - 1° ottobre	Stesura di DOCUMENTAZIONE.md (256 KB), deduplicazione, verifiche, licenze, commit unico e release v1.0	Hermes (skill multiagen t-document-assembly)

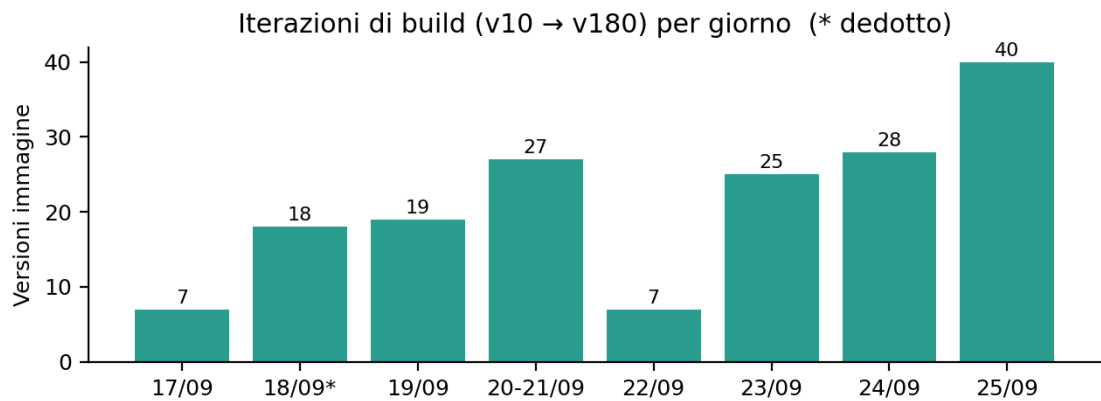


Figura 3 — Versioni di immagine per giorno, dalla tabella cronologica di DOCUMENTAZIONE.md. Il valore del 18/09 è dedotto dal salto v16 → v35.

Il ritmo è intenso: circa 170 versioni in nove giorni, con sessioni documentate anche di notte (00:16-01:23, fino alle 07:15) e una punta di 40 versioni il 25 settembre, dedicate soprattutto a UI e standby del display.

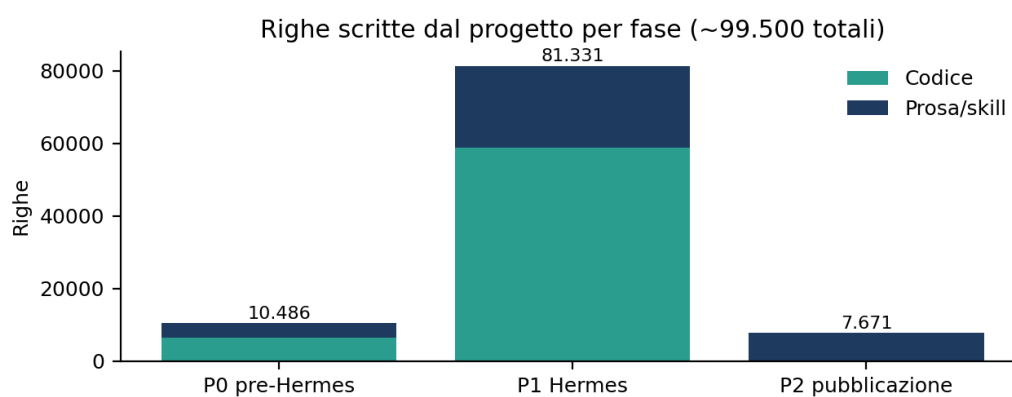


Figura 4 — Righe di codice e prosa scritte dal progetto per fase, escluse le sorgenti di terzi e gli output degli strumenti.

Fase	File scritti	Righe	Firma stilistica
P0 pre-Hermes	67	~10.500	Inglese (22 documenti su 24), 97 emoji 🐞🐛, “Executive Summary”, “Root Cause”
P1 Hermes	460	~81.300	Italiano, 178 marcature FATTO/IPOTESI/DA CONFERMARE, prima persona dell’agente
P2 pubblicazione	4	~7.700	DOCUMENTAZIONE, DEDUPLICA, README, ESTRAZIONE-BLOB

4. Agenti e modelli AI nelle diverse fasi

L'ipotesi di partenza era un'orchestrazione composta solo da Hermes e DeepSeek Pro. L'analisi la conferma per Hermes e la smentisce sull'esclusività: nel lavoro compaiono almeno altri tre modelli, più un agente non identificato nella fase P0.

Agente / modello	Fase	Ruolo	Evidenza nel repo	Stato
Hermes Agent (Nous Research, MIT)	P0b-P2	Orchestratore, profilo kernel-re	Deleghe deleg_*, 7 transcript di subagenti, skill firmate "Hermes Agent"	Confermato
Subagenti Hermes	P1	Fan-out di ricerca da 9-10 agenti, analisi in batch	Transcript task-b1-*/b2-*: 321 tool call in totale, nessun intervento umano	Confermato
DeepSeek (V4 Pro)	?	Ipotesi: modello principale	0 occorrenze in tutto l'albero	Non riscontrato
GPT-6 Astra (OpenAI) via provider "oneprovider"	P1	Consulente: diagnosi del display (timeout RCG), "DESIGN approvato da Astra" per il client DRM persistente	Comando hermes -m gpt-6-astra --provider oneprovider --reasoning max; un consulto sul modem in timeout	Confermato
OpenRouter	P1	Canale alternativo per Astra	Errore crediti HTTP 402, poi vietato esplicitamente	Fallito
gpt-5.6-luna (OpenAI)	P0b (04/09)	Test di configurazione	Prima sessione Hermes registrata: 8 messaggi, nessun lavoro sul dispositivo	Confermato
claude-haiku-4-5	P0b (04/09)	Test di configurazione	Errore 404 model_not_found	Fallito
Modello di visione per vision_analyze	P1	Lettura dei frame della webcam puntata sul telefono	DeepSeek V4 Pro non accetta immagini in input, quindi serve un altro modello	Ignoto
Agente della fase P0	P0	RE del bootloader, prototipi, report in inglese	Stile diverso, ciclo FINDINGS → SOLUTION_FOUND → REALITY_CHECK	Ignoto

4.1 L'impronta della fase P0

La documentazione stessa ammette che il lavoro del 2 luglio - 7 agosto "non è attribuibile a questo profilo Hermes né a uno strumento registrato". Che sia comunque assistito da AI emerge da diversi indizi.

- In bootloader-re/ c'è una sequenza di file del 6 agosto con nomi tipici dell'autocorrezione di un LLM: FINDINGS, SOLUTION_FOUND, REALITY_CHECK, REAL_SOLUTION_DEEP_ANALYSIS, ACTUAL_RESULTS. FINDINGS attribuisce kvm-arm.mode=protected all'ABL; SOLUTION_FOUND lo smentisce (il parametro sarebbe in CONFIG_CMDLINE) senza ritirare il report precedente.
- REALITY_CHECK si rivolge direttamente all'utente ("all your working OpenWrt builds"): è la voce di un assistente in chat.
- hyp_attack.c usa indirizzi commentati come "SM8350 Gunyah addresses" su un dispositivo SM8450: è un errore tipico di codice generato senza verifica.
- FINAL_REPORT dichiara "Framework: reverse-skill + radare2". Reverse-skill è però un pacchetto di skill indipendente dal client, compatibile con Claude Code, Kiro, Cursor, Cline e altri: non identifica lo strumento usato.

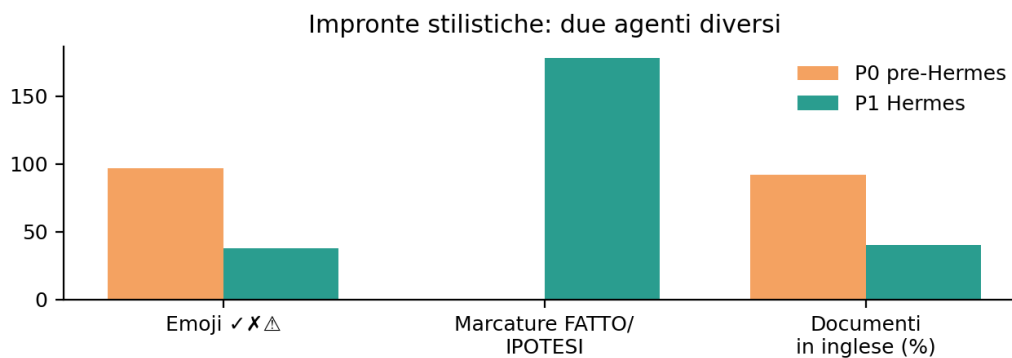


Figura 5 — Marcatori stilistici a confronto tra P0 e P1.

4.2 Un falso positivo da scartare

La stringa “Assisted-by: Claude:claude-sonnet-4-6” compare in `tools/qdlrs/AGENTS.md`. Quel file però è la policy del progetto upstream `qdl-rs` di Qualcomm, incluso come dipendenza: la stringa è l’esempio di formato del trailer “Assisted-by” secondo la convenzione del kernel Linux, non un indizio dell’uso di Claude nel port.

5. Distribuzione dell'intervento umano e AI

Il rapporto tra lavoro umano e lavoro AI cambia molto a seconda di cosa si misura. Ho quindi usato tre metriche distinte.

5.1 L'impronta umana documentata

Il file 08-skill/nx679j-openwrt/references/regole-utente.md raccoglie quasi alla lettera le richieste dell'utente fatte nelle sessioni 16-22 settembre, e annota che erano "vincolanti" perché l'utente "si è ripetutamente irritato quando non applicate". Insieme alle altre citazioni sparse nel repo si arriva a circa una trentina di frasi realmente pronunciate dall'utente. Alcuni esempi rappresentativi:

«ma prima di scrivere altro codice, come ti ho detto 2 milioni di volte, cerca le info online sguinzagliando anche 20 agenti simultaneamente»

«noi dovevamo usare i componenti di OpenWrt standard, non creare un protocollo nostro che non ha nessuna funzione»

«ho collegato una webcam che guarda lo smartphone, perché non trovi online un modo per poterla usare così da avere tu stesso degli input visivi senza il mio supporto?»

«ma adesso tu vuoi dirmi che non puoi entrare ssh né da gadget né da wifi!? [...] lo vedo io connesso il gadget e vedo la rete wifi del telefono!»

Tipo di intervento umano	Esempi documentati
Direzione e metodo	Cercare prima di scrivere codice (nata la skill block-research-fanout); usare componenti OpenWrt standard (rimosso il proto custom); persistenza; Android come "oracolo"; vietato dichiarare falsi successi
Conoscenza dell'ambiente	IP del Wi-Fi 192.168.77.1; procedura fastboot → slot A → Magisk; contestazione della "irraggiungibilità"; correzione di una lettura visiva errata ("linea ciano")
Azioni fisiche	Collegamento e puntamento della webcam; reboot e power-cycle ("i reboot li faccio io", al massimo un'azione utente per test); verifica dei tasti laterali; segnalazione dei glitch
Autorizzazioni	23/09: un solo commit DRM reale consentito, con limiti dichiarati (nessun flash, nessuna modifica al modem)
Scrittura di codice o documenti	Nessuna traccia; anche le skill co-firmate "user, Hermes Agent" sono redatte dall'agente

Lato agentico. I 7 transcript dei subagenti contengono 321 tool call e zero interventi umani, con un ritmo di circa 7 tool call al minuto. L'agente disponeva inoltre di un circuito di verifica visiva autonomo (webcam, OBS e vision_analyze), che riduceva il bisogno di input umano. Quel circuito è nato però da una richiesta esplicita dell'utente.

5.2 Le forchette

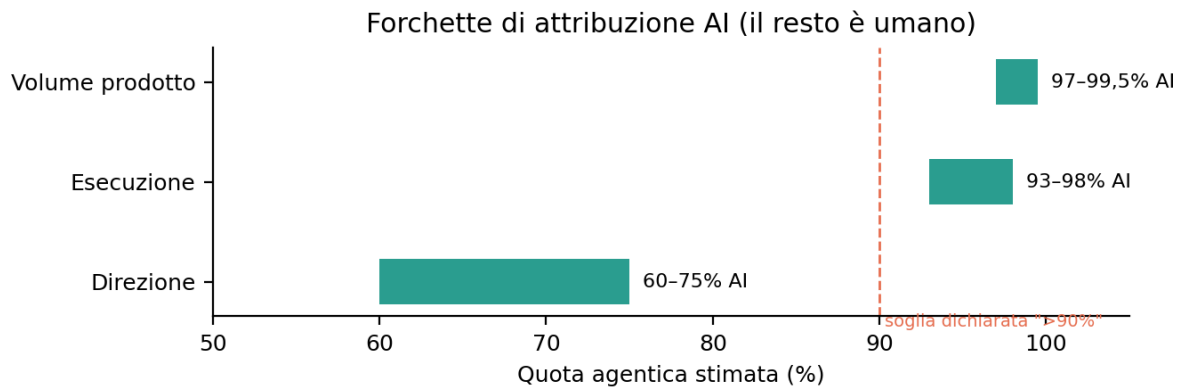


Figura 6 — Quota agentica stimata per metrica; il complemento è la quota umana. La linea tratteggiata indica la soglia "oltre il 90%" dichiarata.

Metrica	Quota AI	Quota umana	Motivazione
Volume prodotto (righe di codice e documenti)	97-99,5%	0,5-3%	~99.500 righe scritte dall'agente; l'apporto umano diretto è quasi solo nelle frasi citate. Margine per eventuali ritocchi manuali non tracciati in P0
Esecuzione (azioni sul sistema e sul dispositivo)	93-98%	2-7%	Migliaia di comandi e tool call agentici contro decine o poche centinaia di azioni fisiche; molti cicli di reboot e cambio slot erano autonomi
Direzione (svolte decisive)	60-75%	25-40%	Diverse svolte metodologiche, l'infrastruttura di osservazione e varie correzioni di fatto vengono dall'utente. Stima qualitativa

Conclusione. Il dato "oltre il 90% agentico" è corretto, anzi prudente, se si riferisce a quanto è stato scritto ed eseguito. Non descrive bene chi ha guidato il progetto: lì il ruolo umano è minoritario ma tutt'altro che marginale. Senza l'utente l'agente si era bloccato in attese passive, aveva riscritto componenti già esistenti e aveva dato per irraggiungibile un dispositivo connesso.

Limiti. Le citazioni conservate sono quelle che l'agente ha scelto di trascrivere, presumibilmente le più incisive, quindi i turni umani reali sono sottostimati. Con state.db la metrica sulla direzione diventerebbe un conteggio.

6. Stima dei costi: tre scenari

Nel repository non ci sono log di consumo, quindi i volumi di token sono ricostruiti dagli artefatti. I prezzi sono invece quelli pubblici in vigore nel settembre-ottobre 2026. Per il loop principale si assume DeepSeek V4 Pro, come da ipotesi iniziale: il modello principale non è dichiarato nel repo, ed è questa la vera incognita della stima.

6.1 Ipotesi di volume (scenario con la totalità delle attività)

Componente	Ipotesi	Base nel repo
Loop principale Hermes	~10.000 turni; contesto medio ~90k token; 90% di cache hit; ~1.500 token in uscita per turno	~180 build, lavoro dal 16 al 30/09 anche notturno; documenti di stato da ~30k token riletto spesso
Subagenti	~175 subagenti × 25 turni; contesto ~35k; 75% di cache hit; ~1.000 token in uscita per turno	Ritmo misurato nei 7 transcript; fan-out da 9-10 agenti; 12 file RICERCA/ESITO; 4 ID di delega
Overhead di orchestrazione	+8% sul loop principale	Compressione del contesto, memoria, curation delle skill. Hermes in sé è gratuito
Visione	~1.000 analisi; ~2.500 token in input e ~300 in output ciascuna	Verifica via webcam “sempre”, con raffiche di 8-10 frame
GPT-6 Astra	5 consulti, di cui 2 riusciti; ~40k in input e ~30k in output ciascuno	reasoning max; 2 consulti completati e 1 in timeout documentati
Fase P0	~13% del volume Hermes	Rapporto tra le righe scritte (10,5k contro 81k)
Totale loop + subagenti	~1,05 miliardi di token in input, ~19 milioni in output	Scenario B e C

Tentativi falliti (scenario A). Stimati in circa il 60% dell’attività: l’ipotesi QMUX falsificata, v101/v102 annullate, v121/v122 mai flashate, la caccia ai boot v104-v112, lo standby v168-v172. Per la fase P0 la quota di fallimento è stimata all’80%, perché la strada kernel upstream/UEFI/hypervisor è stata abbandonata del tutto.

6.2 Prezzi applicati (dollari per milione di token)

Modello	Condizione economica	Condizione cara	Note
DeepSeek V4 Pro	Fuori punta: 0,66 input / 0,022 cache / 1,98 output	Di punta: 1,32 / 0,044 / 3,96	Ore di punta: 01-04 e 06-10 UTC nei giorni feriali; fuori punta -50%
DeepSeek V4 Flash (visione, ipotesi)	0,22 / 0,007 / 0,66	0,44 / 0,014 / 1,32	Il vero modello di visione è ignoto
GPT-6 Astra	Flex/Batch: 5 / 0,50 / 25	Fast (priority): 20 / 2 / 100	Standard 10 / 1 / 50; oltre 272K token di input: 20 / 75
gpt-5.6-luna	Trascurabile	Trascurabile	8 messaggi di test
claude-haiku-4-5, OpenRouter	0	0	Chiamate fallite (404 e 402)

Nello scenario C, oltre ai prezzi di punta e alla modalità Fast, la cache hit è stata ridotta di 10 punti, perché una cache meno efficace fa parte delle condizioni peggiori.

6.3 Risultati

Voce	A. Minimo	B. Medio	C. Massimo
Orchestratore (loop principale)	\$42,77	\$106,92	\$328,68
Overhead Hermes	\$3,42	\$8,55	\$26,29
Subagenti (fan-out e deleghe)	\$14,58	\$36,45	\$92,45
Visione webcam	\$0,30	\$0,75	\$1,50
GPT-6 Astra	\$1,90	\$4,75	\$19,00
gpt-5.6-luna (test)	≈ \$0	≈ \$0	≈ \$0
Fase P0 (agente ignoto)	\$3,73	\$18,64	\$54,75
Totale	≈ \$67	≈ \$176	≈ \$523

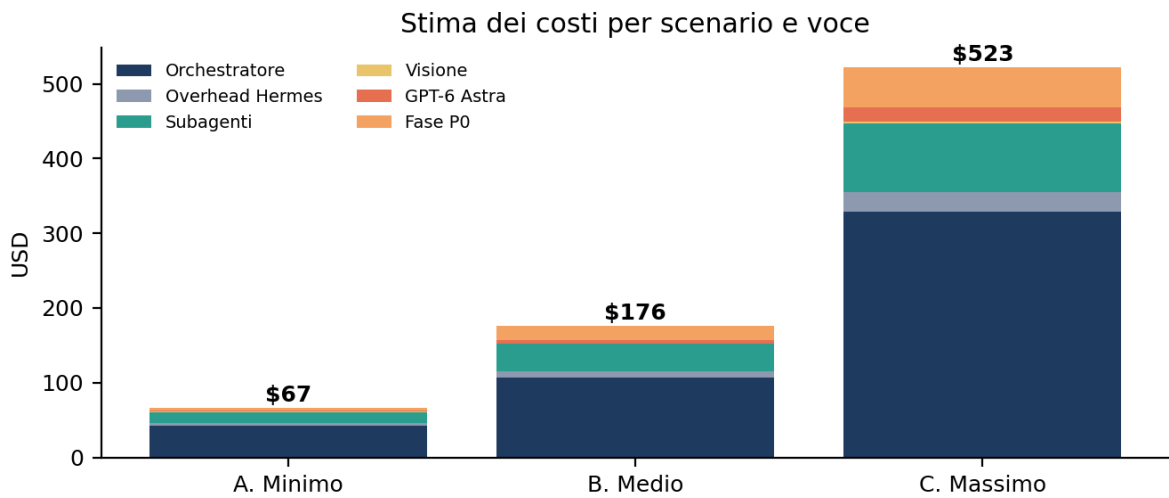


Figura 7 — Costi stimati per scenario e voce.

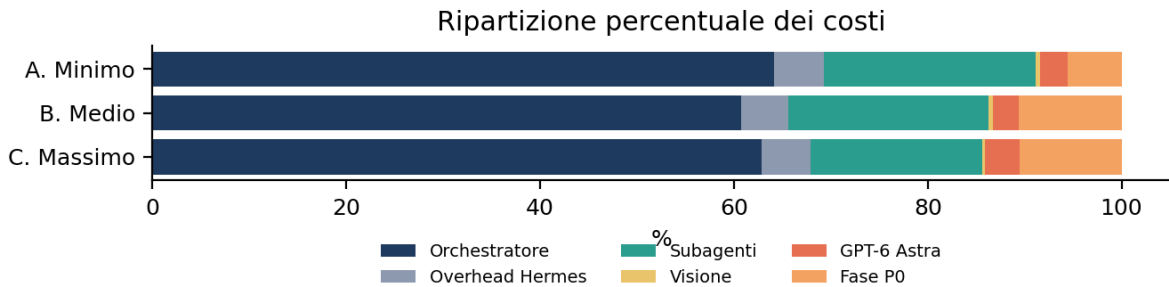


Figura 8 — Ripartizione percentuale: il loop principale vale circa il 60-65%, i subagenti circa il 18-22%, l'orchestrazione circa il 5%.

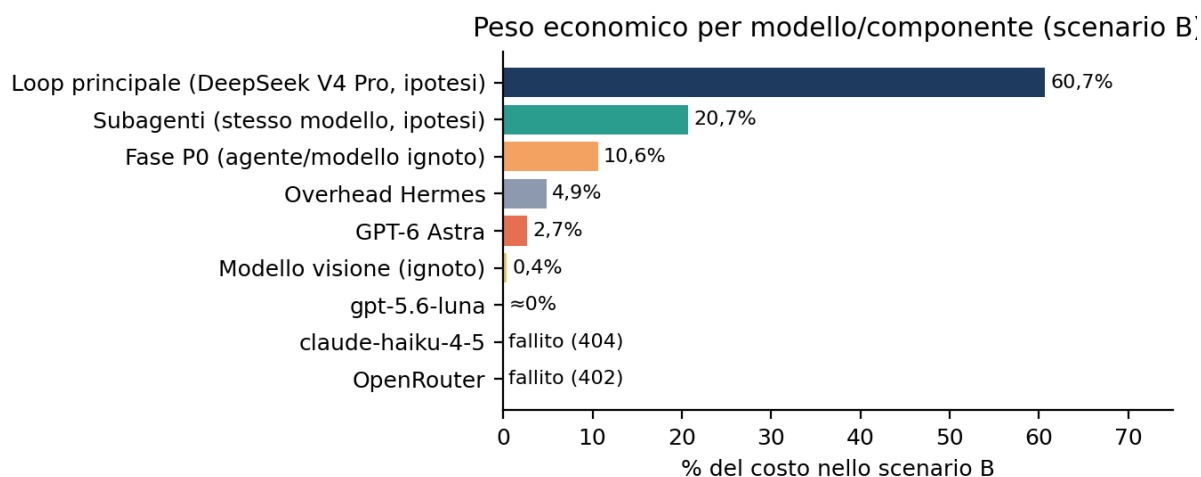


Figura 9 — Peso economico per modello o componente nello scenario B. Astra, pur con un prezzo unitario 15-25 volte più alto, pesa il 2,7% perché è stato usato pochissimo.

6.4 Sensibilità e lettura della forchetta

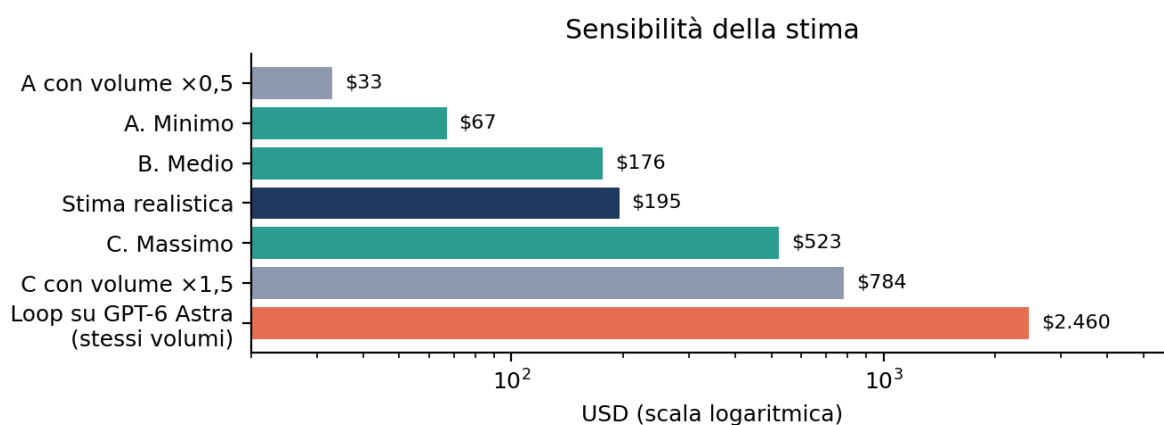


Figura 10 — Sensibilità della stima (scala logaritmica).

- **La stima più realistica è vicina allo scenario B.** Le fasce di punta DeepSeek coprono circa 7 ore su 24 nei giorni feriali, i weekend sono interamente fuori punta, ma alcune sessioni documentate cadono in fascia di punta (07:15-08:25 e 08:28-12:23 ora italiana). Ponderando si arriva a circa 190-200 dollari.
- **Il volume pesa più del prezzo.** Con volumi dimezzati lo scenario A scende a circa 33 dollari; con volumi aumentati del 50% lo scenario C sale a circa 784. La forchetta difendibile è quindi da circa 35 a circa 800 dollari.
- **La variabile decisiva è il modello principale.** Se il loop principale fosse girato su GPT-6 Astra a tariffa standard, con gli stessi volumi, quella sola voce costerebbe circa 2.460 dollari.
- **Costi non quantificati:** il ricarico del provider “oneprovider”, le API di ricerca web e scraping dei fan-out, lo strumento della fase P0 (che potrebbe essere un abbonamento a costo fisso), l’hardware e il tempo umano.

Appendice A — Limiti e passi successivi

- I log di sessione (state.db, request_dump_*.json) sono citati ma non pubblicati: con quei file si potrebbero sostituire tutte le ipotesi di volume con conteggi reali per sessione e per modello, e trasformare in conteggi anche le metriche della sezione 5.
- La datazione per fase usa il percorso, il contenuto e la cartella, perché git non ha storia: errori di assegnazione al confine tra P0 e P1 sono possibili ma marginali.
- La classificazione dei file (terzi, output, codice, prosa) è automatica con regole sui percorsi; i conteggi possono variare di qualche punto percentuale.
- I modelli GPT-6 Astra e gpt-5.6-luna sono reali e documentati pubblicamente; l'identità del provider "oneprovider" e il suo listino non sono verificabili.

Appendice B — Fonti esterne

Fonte	Uso
github.com/Saddytech/openwrt-nx679j e release v1.0	Identità del repository, descrizione e verifiche dichiarate
github.com/CAPP-TESTS/openwrt-nx679j	Copia analizzata (stesso commit f9a92fc)
deepseek.ai/pricing · benchlm.ai/deepseek/api-pricing · layer3labs.io/guides/deepseek-pricing · devtk.ai/en/models/deepseek-v4	Prezzi DeepSeek V4 Pro e Flash, fasce di punta, capacità del modello
eesel.ai/blog/gpt-6-astra-pricing · cloudzero.com/blog/gpt-6-pricing · docs.litellm.ai/blog/gpt_6_astra	Prezzi di GPT-6 Astra (Standard, Flex/Batch, Fast, contesto lungo)
requesty.ai · playground.roboflow.com · cloudprice.net	Esistenza e caratteristiche di gpt-5.6-luna e gpt-6-astra
github.com/zhaoxuya520/reverse-skill	Natura del pacchetto reverse-skill (indipendente dal client)
hermes-agent.nousresearch.com	Hermes Agent: licenza MIT e provider personalizzati

Appendice C — Percorsi chiave nel repository

Percorso	Rilevanza
DOCUMENTAZIONE.md (sezione "Fase 0d", "Da chiarire")	Provenienza delle sessioni, fase pre-Hermes non attribuita
08-skill/nx679j-openwrt/references/regole-utente.md	Citazioni testuali dell'utente
01-documenti/NX679J-NEW-SESSION-PROMPT.md, MODEM-HANDOFF-20260919.md	Uso di GPT-6 Astra via oneprovider, errore 402 di OpenRouter
01-documenti/wifi-luci/STATO-ATTUALE.md	"DESIGN approvato da Astra", autorizzazioni utente del 23/09
09-albero-originale/experiments-extra/refs/session4-20260920/subagents/	Transcript dei subagenti Hermes
05-bootchain-re/bootloader-re/	Documenti della fase P0 (stile, autocorrezioni)
09-albero-originale/hypervisor-bypass/hyp_attack.c	Indirizzi SM8350 su un dispositivo SM8450
02-sorgenti/wifi-luci/build-v54...v90.py	Ultimi builder disponibili, con percorsi assoluti
DEDUPICA.md	1.450 file rimossi e riferimenti a un archivio di revisione non pubblicato